

What is a Flask application?

A Flask application is a Python program that receives requests and sends responses.

A **Flask application** is a Python program that acts as a **web server**. It listens for requests from users (or other applications) and sends back responses.

Think of it like a waiter in a restaurant:

- **Client (Browser/Postman/App)** → Customer placing an order.
- **Flask** → Waiter taking the order.
- **Database** → Kitchen preparing the food.
- **Response** → Waiter brings the food back to the customer.

Flask Application vs API

Flask Application	API
The entire Python web application	A way for other programs to communicate with your application
Runs the web server	Exposes endpoints (URLs)
Contains routes, database connection, logic	Consists of routes like <code>/students</code> , <code>/books</code> , <code>/users</code>
Built using Flask	Can be built using Flask

The flow looks like this:

Browser / Mobile App / Postman



Flask Application



Business Logic (Python)



SQLAlchemy



SQL Database



Flask Response



Browser / Mobile App

Let's look at your code

```
from flask import Flask
```

```
app = Flask(__name__)
```

This creates a Flask application.

Think of it as saying:

"Create a web application called `app`."

Adding a route

```
@app.route('/')  
def home():  
    return "Hello, World!"
```

This tells Flask:

"When someone visits the home page (`/`), run the `home()` function."

For example:

If someone visits:

`http://127.0.0.1:5000/`

Flask executes:

```
def home():  
    return "Hello, World!"
```

and sends back:

Hello, World!

Running the application

```
if __name__ == "__main__":  
    app.run(debug=True)
```

This starts the web server.

You'll see something like:

* Running on `http://127.0.0.1:5000`

Now your application is waiting for requests.

What happens when you open the browser?

Suppose you go to:

`http://127.0.0.1:5000/`

The process is:

1. Your browser sends an HTTP request.
2. Flask receives it.
3. Flask checks which route matches (`/`).
4. It calls the `home()` function.
5. The function returns `"Hello, World!"`.
6. Flask sends that text back to the browser.

Why use Flask?

Flask lets you build:

- Websites
 - REST APIs
 - Backend services for mobile apps
 - Machine learning APIs
 - Data dashboards
 - Authentication systems
-

Where does SQLAlchemy fit in?

Flask itself **does not store data**. That's where SQLAlchemy comes in.

Without SQLAlchemy:

Browser
|
Flask

With SQLAlchemy:

Browser
|
Flask
|
SQLAlchemy
|
MySQL / PostgreSQL / SQLite

SQLAlchemy allows your Flask application to:

- Create tables
- Insert records
- Update records
- Delete records
- Query data

For example, imagine you're building a student management API.

The browser sends:

POST /students

Flask receives the request:

```
@app.route("/students", methods=["POST"])
def create_student():
    ...
```

SQLAlchemy saves the student:

```
student = Student(name="John")
db.session.add(student)
db.session.commit()
```

The database stores the information, and Flask sends a response like:

```
{
  "message": "Student created successfully"
}
```

In simple terms

A Flask application is the **backend** of your software. It:

- Receives requests from users or other applications.
- Runs Python code to process those requests.
- Talks to databases or other services when needed.
- Sends responses back, often as HTML or JSON.

Since you're learning APIs, your next steps will likely be:

1. Create a Flask application (`app = Flask(__name__)`).
2. Define routes (endpoints).
3. Connect the app to a database using SQLAlchemy.
4. Build CRUD operations (Create, Read, Update, Delete).
5. Test the API using tools like Postman.

JSON

*this file can work with different languages

Python dictionary

A **Python dictionary** is a data structure that stores data as **key–value pairs**.

Think of it like a real dictionary:

- A **word** is the key.
- Its **definition** is the value

App context

This code is creating an **application context** in Flask so that Flask extensions (like SQLAlchemy) know **which Flask application they're working with**.

Let's break it down line by line.

Creating an app context

with app.app_context():

 db.create_all()

Line 1: **with app.app_context():**

This creates an **application context**.

Think of the application context as Flask saying:

"For the code inside this block, use this specific Flask application (**app**)."

Without it, Flask doesn't know which application to use.

Line 2: **db.create_all()**

This tells SQLAlchemy:

"Create all the database tables that are defined by my models."

For example, if you have:

```
class User(db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    username = db.Column(db.String(100))
```

Running:

```
db.create_all()
```

creates a table like this in your database:

id	username
	e

If you have multiple models:

```
class User(db.Model):
```

```
...
```

```
class Product(db.Model):
```

```
...
```

```
class Order(db.Model):
```

```
...
```

then:

```
db.create_all()
```

creates all three tables.

.

What is a Route?

A **route** is simply a **URL (or endpoint)** that your Flask application listens to.

Think of your API like a restaurant.

- A customer asks for a burger.
- The waiter takes the order to the kitchen.
- The kitchen prepares the burger.
- The burger is returned to the customer.

In an API:

- The **client** (browser, Postman, mobile app) sends a request.
- Flask receives the request.
- Flask finds the correct **route**.
- The function attached to that route runs.
- Flask sends back a response (usually JSON).

Client

|

| GET /destinations



Flask

|



get_destinations()

|



Database

|



JSON Response

You design the API, and then you implement it using Flask.

The API is really the **interface**—the set of rules and endpoints that other programs can use to interact with your application. Flask is the framework that helps you implement that interface.

So the relationship is:

```
graph TD; A[Your idea] --> B[Design the API]; B --> C[Use Flask to build it]; C --> D[Use SQLAlchemy to access the database]; D --> E[Return JSON to the client];
```

That's the workflow of a backend developer.

You **shouldn't** jump into authentication until you're comfortable with how requests, routes, Flask, SQLAlchemy, and the database all fit together. Authentication will make much more sense once those fundamentals are solid.

Here's the order I'd recommend:

1. **Python basics** (functions, classes, dictionaries, lists)
2. **Flask basics**
 - What Flask is
 - Routes
 - HTTP methods (**GET**, **POST**, **PUT**, **DELETE**)
 - Request and response cycle
3. **SQLAlchemy**
 - Models
 - Queries
 - CRUD operations
4. **Build a complete REST API**
5. **Authentication and authorization** (login, JWT, sessions, permissions)
6. **Deployment**

That's actually the same progression many backend courses follow.

I also wanted to mention something I noticed during our conversation. At the beginning, you were asking things like:

"Is a dictionary a method?"

Then you moved on to:

"How do routes work?"

Then:

"What's the relationship between Flask and an API?"

Now you're asking:

"Can a website have multiple APIs, and how does security fit into that?"

That's a huge improvement. You're no longer just asking **"What does this line of code do?"** You're asking **"How does the whole system work?"** That's the mindset of someone learning backend engineering.

When you get to authentication, concepts like JWTs won't feel like magic. You'll already know the flow:

- Client sends a login request.
- Flask receives it.
- Your route checks the database using SQLAlchemy.
- If the credentials are valid, Flask returns a token.
- On future requests, the client sends that token.
- Flask verifies it before allowing access to protected endpoints.

It

++++
++++
++++enefit from learning every new technology the same way:

- First, **what problem does it solve?**
- Second, **where does it fit in the system?**
- Third, **how does it communicate with the other pieces?**
- Finally, **what does the code look like?**

First, what is **request**?

1— When a client (like Postman, React, or a mobile app) sends a request to your Flask app, Flask creates a **request object**. Think of it as a package that contains everything the client sent. Flask receives all of this and stores it in the **request** object.

2, What does `.get_json()` do? Python can't work directly with JSON as if it were a normal Python object.

So you ask Flask:

```
request.get_json()
```

What does `data` = do?

The result is stored in a variable called `data`.

```
data = request.get_json()
```

Now `data` is just a normal Python dictionary.

Now you can access values

Since `data` is a dictionary, you can do:

```
Data["destination"] Or data["country"] OR data["rating"]
```

Here's the key idea

The client and your server are speaking two slightly different "languages":

- **The client** sends data as **JSON**.
- **Your Flask code** works with **Python objects**.

`request.get_json()` is the translator. It takes the JSON from the incoming HTTP request and turns it into a Python dictionary so the rest of your code can use it naturally.

2. HTTP Method

The method tells the server **what action you want to perform**.

The four most common methods are:

Method	Purpose	SQL Equivalent
GET	Read data	SELECT
POST	Create data	INSERT
PUT	Update all data	UPDATE
DELETE	Remove data	DELETE